

PS2 - Solutions

Paolo Ciracì, Enrico Didoli, Niccolò Disassonia

2024-05-09

Exercise 1

```
psych<-read.table("data/psych.txt",header=T)
psych$Sex <- factor(psych$Sex)
psych <- psych[, -1] #removing index variable
```

Question 1.1

We first of all select only the rows of interest and then estimate the factors: the first time for $m = 5$ and the second time for $m = 6$.

```
psych.grant <- psych[psych$group == "GRANT", -27] #removing 'group' variable
library(psych)
```

We will estimate the model removing also the variables Sex and Age, since they are not object of our analysis.

```
fa.ml <- fa(r = psych.grant[, -c(1, 2)], nfactors = 5, rotate = 'none', fm = 'ml'); fa.ml
```

```
## Factor Analysis using method = ml
## Call: fa(r = psych.grant[, -c(1, 2)], nfactors = 5, rotate = "none",
##      fm = "ml")
## Standardized loadings (pattern matrix) based upon correlation matrix
##      ML1  ML2  ML3  ML4  ML5  h2  u2 com
## V1  0.55  0.00  0.47 -0.15  0.00  0.55  0.45  2.1
## V2  0.34 -0.03  0.29 -0.06  0.13  0.22  0.78  2.3
## V3  0.37 -0.14  0.43 -0.10  0.04  0.35  0.65  2.4
## V4  0.46 -0.10  0.30 -0.11  0.15  0.35  0.65  2.2
## V5  0.72 -0.25 -0.22 -0.08  0.00  0.64  0.36  1.5
## V6  0.72 -0.37 -0.17 -0.01 -0.15  0.71  0.29  1.7
## V7  0.73 -0.34 -0.23 -0.13  0.01  0.71  0.29  1.7
## V8  0.69 -0.14 -0.04 -0.11  0.08  0.52  0.48  1.2
## V9  0.72 -0.42 -0.20  0.02 -0.02  0.74  0.26  1.8
## V10 0.52  0.60 -0.38  0.04  0.12  0.79  0.21  2.8
## V11 0.57  0.35 -0.02  0.06 -0.37  0.59  0.41  2.5
## V12 0.49  0.54  0.01 -0.12  0.13  0.56  0.44  2.2
## V13 0.63  0.35  0.20 -0.38 -0.21  0.75  0.25  2.8
## V14 0.39  0.00  0.06  0.37 -0.24  0.35  0.65  2.7
## V15 0.35  0.03  0.13  0.37 -0.13  0.29  0.71  2.5
## V16 0.46  0.02  0.38  0.28 -0.09  0.43  0.57  2.7
## V17 0.45  0.13  0.03  0.44 -0.11  0.43  0.57  2.3
## V18 0.47  0.25  0.22  0.26  0.02  0.40  0.60  2.6
## V19 0.42  0.05  0.14  0.20 -0.07  0.24  0.76  1.8
## V20 0.60 -0.17  0.18  0.15  0.23  0.49  0.51  1.8
```

```

## V21 0.57  0.23  0.15  0.03  0.16 0.43 0.57 1.6
## V22 0.59 -0.14  0.18  0.13  0.10 0.43 0.57 1.5
## V23 0.67 -0.06  0.21  0.03  0.24 0.55 0.45 1.5
## V24 0.66  0.19 -0.13  0.15  0.13 0.52 0.48 1.4
##
##
##          ML1  ML2  ML3  ML4  ML5
## SS loadings          7.58 1.67 1.32 0.96 0.54
## Proportion Var        0.32 0.07 0.05 0.04 0.02
## Cumulative Var        0.32 0.39 0.44 0.48 0.50
## Proportion Explained  0.63 0.14 0.11 0.08 0.04
## Cumulative Proportion 0.63 0.77 0.88 0.96 1.00
##
## Mean item complexity = 2.1
## Test of the hypothesis that 5 factors are sufficient.
##
## df null model = 276 with the objective function = 11.43 with Chi Square = 1545.24
## df of the model are 166 and the objective function was 1.4
##
## The root mean square of the residuals (RMSR) is 0.04
## The df corrected root mean square of the residuals is 0.05
##
## The harmonic n.obs is 145 with the empirical chi square 106.36 with prob < 1
## The total n.obs was 145 with Likelihood Chi Square = 184.41 with prob < 0.16
##
## Tucker Lewis Index of factoring reliability = 0.975
## RMSEA index = 0.027 and the 90 % confidence intervals are 0 0.048
## BIC = -641.73
## Fit based upon off diagonal values = 0.99
## Measures of factor score adequacy
##
##          ML1  ML2  ML3  ML4  ML5
## Correlation of (regression) scores with factors 0.97 0.92 0.87 0.82 0.74
## Multiple R square of scores with factors        0.95 0.84 0.76 0.66 0.55
## Minimum correlation of possible factor scores    0.90 0.69 0.52 0.33 0.10

```

We now compute the proportion of variance explained by each factor, which is given by the following:

$$\text{Proportion of variance explained by factor } k = \sum_{j=1}^p \frac{\hat{l}_{jk}^2}{\text{trace}(\mathbf{R})}$$

where $\hat{l}_{jk}$ is the estimated j, k th element of the loadings matrix $\hat{\mathbf{L}}$ of dimensions $p \times m$ and $\mathbf{R}$ is the correlation matrix. We conveniently store these values in a vector and then plot them.

```

#The method 'values' of 'fa.ml' gives us the eigenvalues of the estimated R matrix
prop.var.m5 <- 0
for (i in 1:5) prop.var.m5[i] <- sum(fa.ml$loadings[, i]^2)/sum(fa.ml$values)
prop.var.m5

```

```
## [1] 0.62834069 0.13876264 0.10907598 0.07947399 0.04434671
```

```

#Cumulative proportion of variance explained
cum.prop.var.m5 <- cumsum(prop.var.m5)

```

```

#Plotting
par(mfrow = c(1, 2))
plot(prop.var.m5, type = 'b', ylab = 'Proportion of variance explained',
     xlab = 'Factor')

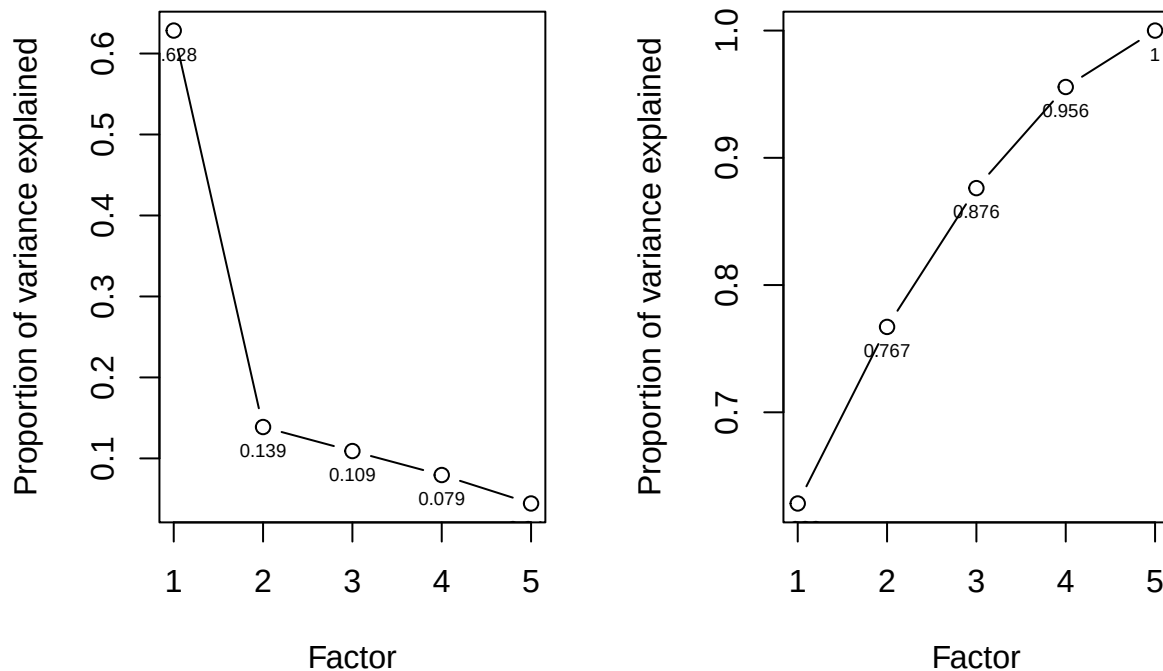
```

```

text(x = 1:5, y = prop.var.m5, round(prop.var.m5, 3), cex = .6, pos = 1)
plot(cum.prop.var.m5, type = 'b', ylab = 'Proportion of variance explained',
     xlab = 'Factor')
text(x = 1:5, y = cum.prop.var.m5, round(cum.prop.var.m5, 3), cex = .6, pos = 1)
title('Proportion and cum. proportion of variance explained by each factor', outer = T,
     line = -2)

```

Proportion and cum. proportion of variance explained by each factor



Now we get the specific variances ψ , which are stored in the diagonal matrix $\hat{\Psi}$ of dimensions $p \times p$. The j th specific variance is given by the difference between the j th element of the correlation matrix $\mathbf{R}$ and the j th communality of the matrix $\hat{\mathbf{L}}$, i.e.:

$$\hat{\psi}_j = r_{jj} - \sum_{k=1}^m \hat{l}_{jk}^2$$

In R we recall the communalities by the output of the `fa()` function we used to perform factor analysis.

#Specific variances:

```

R <- cor(psych.grant[, -c(1, 2)])
spec.var.m5 <- diag(R) - fa.ml$communality; spec.var.m5

```

```

##      V1      V2      V3      V4      V5      V6      V7      V8
## 0.4526230 0.7766271 0.6456081 0.6477157 0.3572527 0.2907273 0.2862728 0.4811694
##      V9      V10     V11     V12     V13     V14     V15     V16
## 0.2573420 0.2082320 0.4133531 0.4360786 0.2525179 0.6488539 0.7117645 0.5653882
##      V17     V18     V19     V20     V21     V22     V23     V24
## 0.5723769 0.5960311 0.7607602 0.5086426 0.5694696 0.5682680 0.4474438 0.4798633

```

We now compute the residual matrix as follows:

$$\hat{\mathbf{R}} = \mathbf{R} - (\hat{\mathbf{L}}\hat{\mathbf{L}}^T + \hat{\mathbf{\Psi}})$$

```
#Residual matrix
R <- cor(psych.grant[, -c(1, 2)])
L <- fa.ml$loadings[, ]
R.tilde <- L%*%t(L) + diag(spec.var.m5)
Residual <- R - R.tilde
```

Notice we could also simply recall the residual matrix from the output of the `fa()` function; however, this matrix will be computed as follows:

$$\hat{\mathbf{R}} = \mathbf{R} - (\hat{\mathbf{L}}\hat{\mathbf{L}}^T)$$

Hence, it won't subtract the specific variances $\hat{\mathbf{\Psi}}$ from the correlation matrix. Moreover, we won't print it since it's too big.

```
#fa.ml$residual
```

To assess the accuracy of the approximation we can compute the sum of squared entries of the residual matrix:

```
sse.m5 <- tr(Residual%*%t(Residual))
#or:
sum(Residual^2)
```

```
## [1] 0.733505
```

We now repeat the above steps to perform factor analysis estimating $m = 6$ factors.

```
#Estimating
fa.ml <- fa(r = psych.grant[, -c(1, 2)], nfactors = 6, rotate = 'none', fm = 'ml')
fa.ml
```

```
## Factor Analysis using method = ml
## Call: fa(r = psych.grant[, -c(1, 2)], nfactors = 6, rotate = "none",
##      fm = "ml")
## Standardized loadings (pattern matrix) based upon correlation matrix
##      ML1  ML2  ML3  ML4  ML5  ML6  h2  u2 com
## V1  0.55  0.00  0.46 -0.20 -0.06  0.03  0.55  0.45  2.3
## V2  0.34 -0.03  0.30 -0.16  0.07  0.23  0.29  0.71  3.4
## V3  0.37 -0.14  0.44 -0.11  0.03 -0.23  0.42  0.58  2.9
## V4  0.46 -0.11  0.30 -0.13  0.13 -0.09  0.36  0.64  2.4
## V5  0.72 -0.26 -0.22 -0.07 -0.03  0.09  0.65  0.35  1.5
## V6  0.72 -0.37 -0.16  0.03 -0.15  0.00  0.71  0.29  1.7
## V7  0.73 -0.35 -0.23 -0.09  0.02 -0.15  0.75  0.25  1.8
## V8  0.70 -0.15 -0.04 -0.10  0.09 -0.21  0.57  0.43  1.4
## V9  0.73 -0.42 -0.18  0.05 -0.03  0.09  0.75  0.25  1.8
## V10 0.51  0.59 -0.39 -0.02  0.16  0.03  0.78  0.22  2.9
## V11 0.58  0.39 -0.04  0.08 -0.42  0.13  0.69  0.31  2.8
## V12 0.48  0.54 -0.01 -0.17  0.13 -0.10  0.57  0.43  2.4
## V13 0.62  0.33  0.15 -0.36 -0.23 -0.14  0.71  0.29  2.9
## V14 0.40  0.03  0.08  0.35 -0.13  0.01  0.31  0.69  2.3
## V15 0.35  0.06  0.15  0.33 -0.04  0.10  0.27  0.73  2.6
## V16 0.46  0.06  0.39  0.21 -0.04  0.08  0.41  0.59  2.5
## V17 0.47  0.18  0.07  0.57  0.01 -0.26  0.65  0.35  2.6
## V18 0.48  0.28  0.23  0.22  0.07  0.01  0.41  0.59  2.7
## V19 0.42  0.07  0.15  0.18 -0.03 -0.02  0.24  0.76  1.7
```

```

## V20 0.60 -0.16 0.20 0.07 0.23 0.09 0.49 0.51 1.8
## V21 0.57 0.23 0.15 -0.10 0.14 0.22 0.48 0.52 2.0
## V22 0.60 -0.12 0.20 0.09 0.07 0.17 0.45 0.55 1.6
## V23 0.66 -0.06 0.23 -0.04 0.23 0.07 0.55 0.45 1.5
## V24 0.66 0.19 -0.11 0.08 0.16 0.07 0.51 0.49 1.4
##
##
## ML1 ML2 ML3 ML4 ML5 ML6
## SS loadings 7.60 1.71 1.35 1.00 0.51 0.42
## Proportion Var 0.32 0.07 0.06 0.04 0.02 0.02
## Cumulative Var 0.32 0.39 0.44 0.49 0.51 0.52
## Proportion Explained 0.60 0.14 0.11 0.08 0.04 0.03
## Cumulative Proportion 0.60 0.74 0.85 0.93 0.97 1.00
##
## Mean item complexity = 2.2
## Test of the hypothesis that 6 factors are sufficient.
##
## df null model = 276 with the objective function = 11.43 with Chi Square = 1545.24
## df of the model are 147 and the objective function was 1.2
##
## The root mean square of the residuals (RMSR) is 0.03
## The df corrected root mean square of the residuals is 0.05
##
## The harmonic n.obs is 145 with the empirical chi square 87.29 with prob < 1
## The total n.obs was 145 with Likelihood Chi Square = 157.38 with prob < 0.26
##
## Tucker Lewis Index of factoring reliability = 0.984
## RMSEA index = 0.021 and the 90 % confidence intervals are 0 0.046
## BIC = -574.2
## Fit based upon off diagonal values = 0.99
## Measures of factor score adequacy
##
## ML1 ML2 ML3 ML4 ML5
## Correlation of (regression) scores with factors 0.98 0.92 0.87 0.84 0.76
## Multiple R square of scores with factors 0.95 0.85 0.76 0.70 0.58
## Minimum correlation of possible factor scores 0.91 0.70 0.52 0.40 0.17
##
## ML6
## Correlation of (regression) scores with factors 0.70
## Multiple R square of scores with factors 0.49
## Minimum correlation of possible factor scores -0.02

#Proportion of variance explained
prop.var.m6 <- 0
for (i in 1:6) prop.var.m6[i] <- sum(fa.ml$loadings[, i]^2)/sum(fa.ml$values)
prop.var.m6

## [1] 0.60391592 0.13558097 0.10735508 0.07944108 0.04040194 0.03330502

#Cumulative proportion of variance explained
cum.prop.var.m6 <- cumsum(prop.var.m6)

#Plotting
par(mfrow = c(1, 2))
plot(prop.var.m6, type = 'b', ylab = 'Proportion of variance explained',
     xlab = 'Factor')
text(x = 1:6, y = prop.var.m6, round(prop.var.m6, 3), cex = .6, pos = 1)
plot(cum.prop.var.m6, type = 'b', ylab = 'Proportion of variance explained',

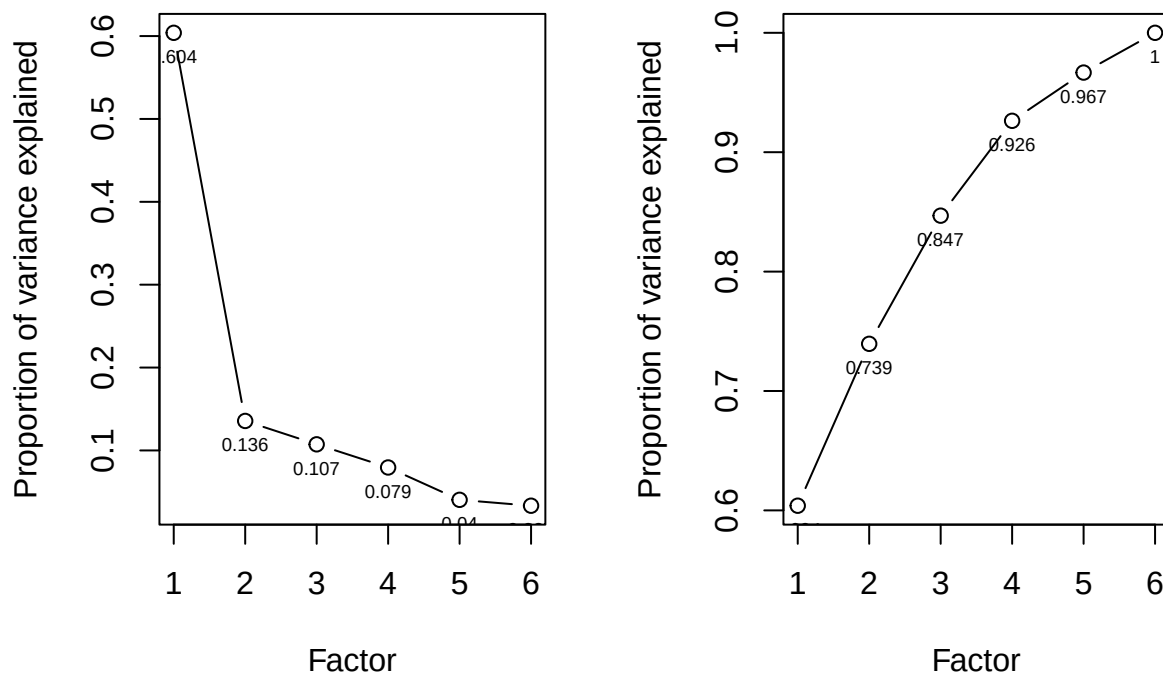
```

```

xlab = 'Factor')
text(x = 1:6, y = cum.prop.var.m6, round(cum.prop.var.m6, 3), cex = .6, pos = 1)
title('Proportion and cum. proportion of variance explained by each factor', outer = T,
      line = -2)

```

Proportion and cum. proportion of variance explained by each factor



```

#Specific variances:
spec.var.m6 <- diag(R) - fa.ml$communality; spec.var.m6

```

```

##      V1      V2      V3      V4      V5      V6      V7      V8
## 0.4474295 0.7098058 0.5771218 0.6433540 0.3459772 0.2946300 0.2519167 0.4288424
##      V9      V10     V11     V12     V13     V14     V15     V16
## 0.2481142 0.2164357 0.3111328 0.4261582 0.2885031 0.6925370 0.7319810 0.5863717
##      V17     V18     V19     V20     V21     V22     V23     V24
## 0.3471880 0.5857354 0.7583078 0.5127850 0.5232765 0.5491420 0.4494492 0.4852705

```

```

#Residual matrix:
L <- fa.ml$loadings[, ]
R.tilde <- L%*%t(L) + diag(spec.var.m6)
Residual <- R - R.tilde

#Sum of squared entries:
sse.m6 <- tr(Residual%*%t(Residual)); sse.m6

```

```
## [1] 0.602021
```

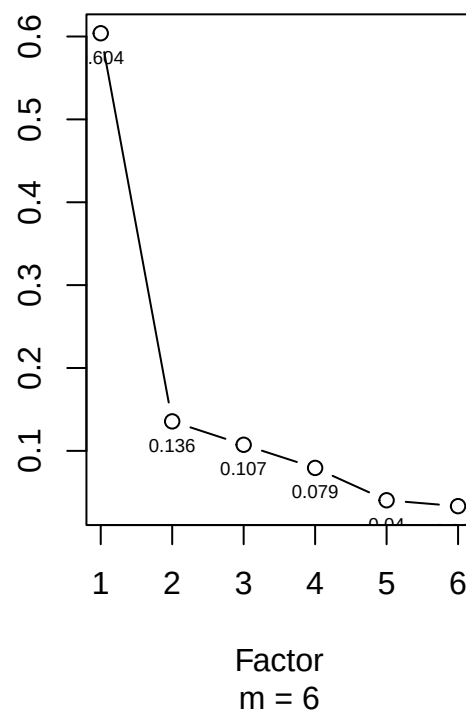
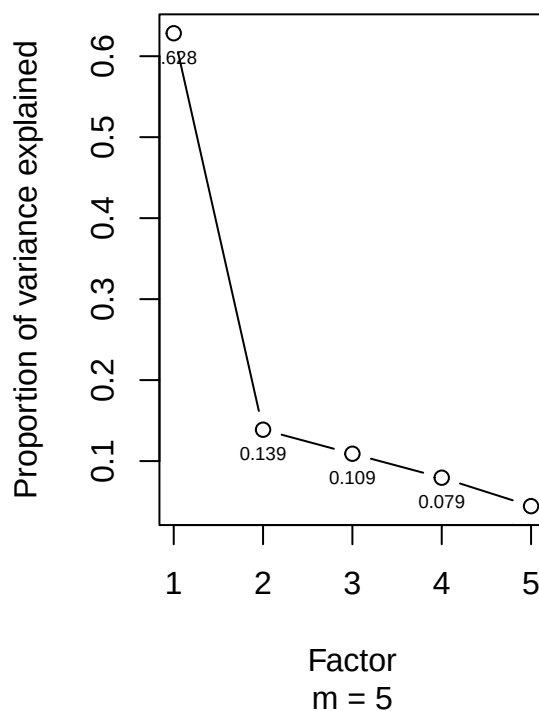
We now plot the proportion of variance explained and the cumulative ones one by one for $m = 5$ and $m = 6$ factors to make a comparison:

```

#Summing-up Plot
par(mfrow = c(1, 2))
#Proportion of variance exp.
plot(prop.var.m5, type = 'b', ylab = 'Proportion of variance explained', xlab = 'Factor',
      sub = 'm = 5')
text(1:5, prop.var.m5, round(prop.var.m5, 3), cex = .6, pos = 1)
plot(prop.var.m6, type = 'b', ylab = '', xlab = 'Factor',
      sub = 'm = 6')
text(1:6, prop.var.m6, round(prop.var.m6, 3), cex = .6, pos = 1)
title('Proportion of variance explained by each of the m factors', outer = T, line = -2)

```

Proportion of variance explained by each of the m factors

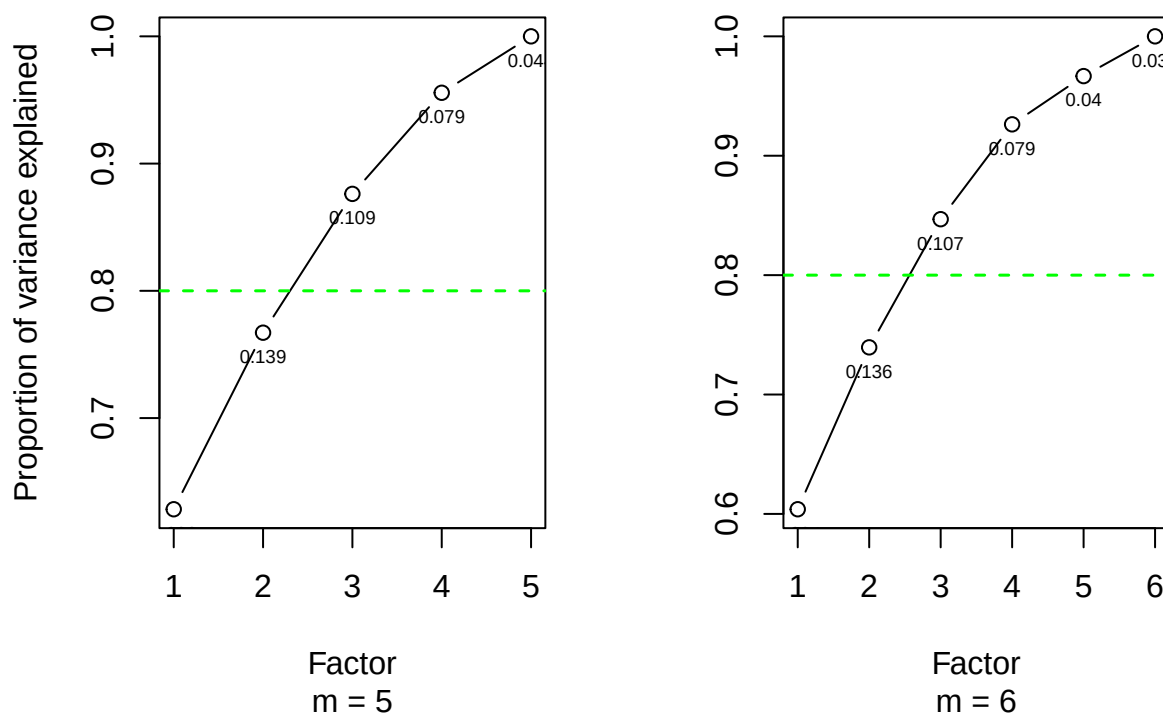


```

#Cumulative proportion of variance exp.
plot(cum.prop.var.m5, type = 'b', ylab = 'Proportion of variance explained',
      xlab = 'Factor', sub = 'm = 5')
text(1:5, cum.prop.var.m5, round(prop.var.m5, 3), cex = .6, pos = 1)
abline(h = .8, lwd = 1.5, lty = 'dashed', col = 'green')
plot(cum.prop.var.m6, type = 'b', ylab = '', xlab = 'Factor',
      sub = 'm = 6')
text(1:6, cum.prop.var.m6, round(prop.var.m6, 3), cex = .6, pos = 1)
abline(h = .8, lwd = 1.5, lty = 'dashed', col = 'green')
title('Cumulative proportion of variance explained by each of the m factors', outer = T,
      line = -2)

```

Cumulative proportion of variance explained by each of the m factors



From the plots, we notice very similar results for both the FA with $m = 5$ and $m = 6$ factors; in both cases we obtain a great reduction in the dimensionality of the data. Since the results are so similar, we'd prefer the model with $m = 5$ factors only because of the parsimony principle.

Question 1.2

We now estimate $m = 5$ factors adding also varimax rotation:

```
fa.ml <- fa(R <- cor(psych.grant[, -c(1, 2)]), nfactors = 5, rotate = 'varimax',
           fm = 'ml')
fa.ml
```

```
## Factor Analysis using method = ml
## Call: fa(r = R <- cor(psych.grant[, -c(1, 2)]), nfactors = 5, rotate = "varimax",
##       fm = "ml")
## Standardized loadings (pattern matrix) based upon correlation matrix
##      ML1  ML3  ML2  ML4  ML5  h2  u2 com
## V1  0.17  0.65  0.13  0.18  0.21  0.55  0.45  1.6
## V2  0.11  0.44  0.09  0.10  0.00  0.22  0.78  1.3
## V3  0.13  0.56 -0.05  0.11  0.09  0.35  0.65  1.3
## V4  0.23  0.53  0.09  0.08  0.01  0.35  0.65  1.5
## V5  0.74  0.19  0.19  0.15  0.05  0.64  0.36  1.4
## V6  0.77  0.19  0.03  0.25  0.12  0.71  0.29  1.4
## V7  0.80  0.21  0.14  0.09  0.05  0.71  0.29  1.2
## V8  0.57  0.34  0.24  0.13  0.04  0.52  0.48  2.2
## V9  0.81  0.20  0.03  0.22 -0.01  0.74  0.26  1.3
## V10 0.18 -0.11  0.85  0.18  0.03  0.79  0.21  1.2
```

```

## V11 0.20 0.07 0.42 0.44 0.42 0.59 0.41 3.4
## V12 0.03 0.23 0.69 0.10 0.13 0.56 0.44 1.3
## V13 0.19 0.43 0.48 0.08 0.54 0.75 0.25 3.2
## V14 0.18 0.06 0.04 0.55 0.08 0.35 0.65 1.3
## V15 0.10 0.12 0.06 0.51 0.00 0.29 0.71 1.2
## V16 0.07 0.41 0.06 0.51 0.05 0.43 0.57 2.0
## V17 0.15 0.07 0.21 0.59 -0.03 0.43 0.57 1.4
## V18 0.03 0.30 0.32 0.46 0.00 0.40 0.60 2.6
## V19 0.16 0.22 0.14 0.38 0.05 0.24 0.76 2.4
## V20 0.37 0.46 0.13 0.29 -0.19 0.49 0.51 3.3
## V21 0.17 0.40 0.43 0.24 0.00 0.43 0.57 2.9
## V22 0.36 0.42 0.11 0.32 -0.07 0.43 0.57 3.1
## V23 0.36 0.54 0.25 0.23 -0.11 0.55 0.45 2.8
## V24 0.37 0.18 0.50 0.32 -0.07 0.52 0.48 3.0
##
##
## ML1 ML3 ML2 ML4 ML5
## SS loadings 3.64 2.96 2.45 2.39 0.63
## Proportion Var 0.15 0.12 0.10 0.10 0.03
## Cumulative Var 0.15 0.27 0.38 0.48 0.50
## Proportion Explained 0.30 0.25 0.20 0.20 0.05
## Cumulative Proportion 0.30 0.55 0.75 0.95 1.00
##
## Mean item complexity = 2
## Test of the hypothesis that 5 factors are sufficient.
##
## df null model = 276 with the objective function = 11.43
## df of the model are 166 and the objective function was 1.4
##
## The root mean square of the residuals (RMSR) is 0.04
## The df corrected root mean square of the residuals is 0.05
##
## Fit based upon off diagonal values = 0.99
## Measures of factor score adequacy
##
## ML1 ML3 ML2 ML4 ML5
## Correlation of (regression) scores with factors 0.93 0.88 0.91 0.84 0.77
## Multiple R square of scores with factors 0.87 0.77 0.83 0.70 0.60
## Minimum correlation of possible factor scores 0.73 0.54 0.66 0.40 0.20

```

First of all, we notice that after the rotation, the order in the factors 2 and 3 changes: this is due to the fact that, after rotation, factor 3 explains a bigger part of the variance (0.25) than factor 2 (0.20). We consider for each factor loadings the ones that are bigger than 0.5 in order to be considered.

- Starting from ML1, we notice that the loadings that are above 0.5 are: V5 to V9, which correspond to: V5 general information, V6 paragraph comprehension, V7 sentence completion, V8 word classification, V9 word meaning. Thus we can conclude that this factor is the *text comprehension* factor;
- Looking at ML3 (second column), we select as for ML1 the following variables: V1, V3, V4, V23, which correspond to the tests: V1 visual perception, V3 paper form board, V4 flags, V23 series completion; thus, we can look at this factor as the one that explains the *creative capabilities* of the individual;
- In ML2 (third column), we select: V10, V12, V24, which correspond to: V10 addition, V12 counting dots, V24 arithmetic problems; so we can look at this factor as the one for the *mathematical and logical capabilities* of the individual: indeed, even if is not above 0.5, we could also notice that V11 and V21 have loadings above 0.4 and correspond respectively to code and numerical puzzles;
- In ML4, we consider: V14 to V17, which correspond to V14 word recognition, V15 number recognition, V16 figure recognition, V17 object-number; so we can view this variable as one that accounts for the *distinction and pattern recognition capabilities* of the individual;

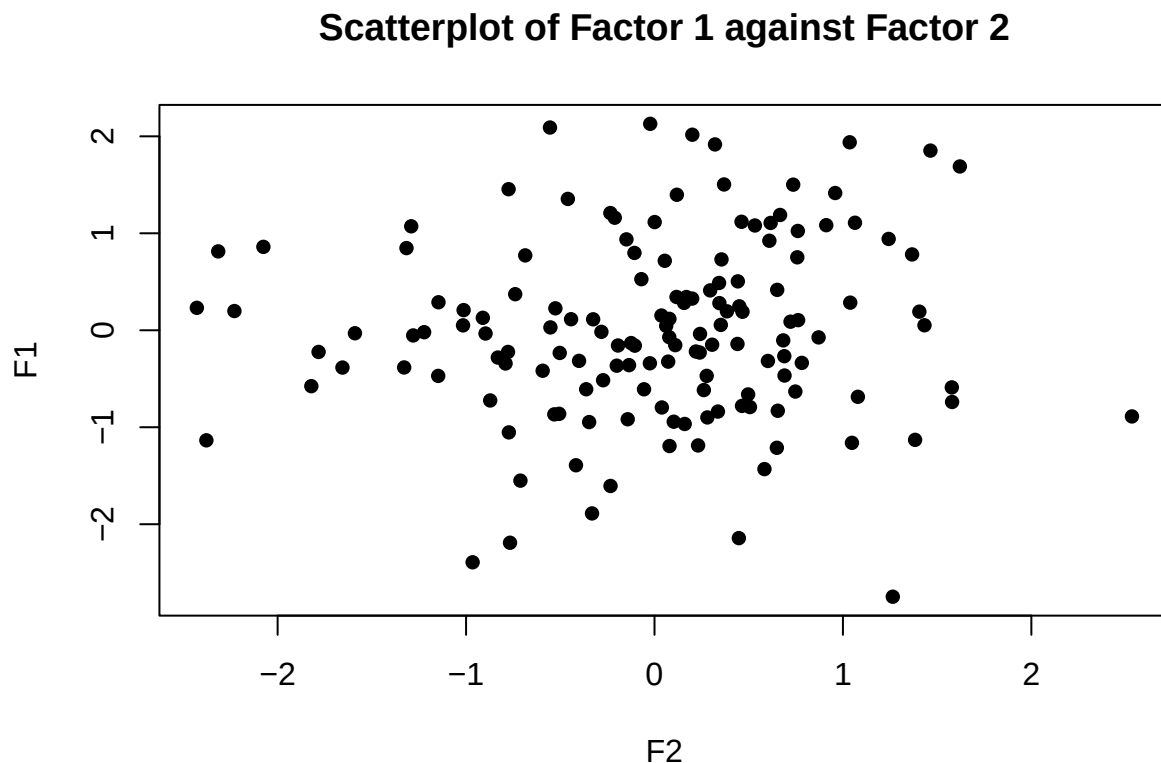
- Finally, looking at ML5, we select only V13 above 0.5. We can then also add V11 (above 0.4), just in order to get a better understanding of what this factor could measure. V11 is code and V13 is straight-curved capitals. Thus, we could argue that this one is a factor which measure the *analytical capabilities of the individual*, even if it is not clear what precisely it could measure, as it was clear for the other factors. We can however safely conclude about the meaning of this factor that it can be interpreted as should be interpreted the straight-curved capitals psychological test.

Question 1.3

Now we plot the first two factor scores for the $m = 5$ solution obtained by the regression method. Notice that since we performed FA using the raw data, using `fa.ml$scores` we will directly get them without using `factor.scores()`.

```
#Estimating and extracting scores
fa.ml <- fa(r = psych.grant[, -c(1, 2)], nfactors = 5, rotate = 'varimax', fm = 'ml')
FS <- fa.ml$scores
F1 <- FS[, 1]
F2 <- FS[, 2]

#Plotting
par(mfrow = c(1, 1))
plot(F1 ~ F2, pch = 16, main = 'Scatterplot of Factor 1 against Factor 2')
```



Now we extract the covariance between the first and the second factor scores:

```
cor(F1, F2)
```

```
## [1] 0.07425159
```

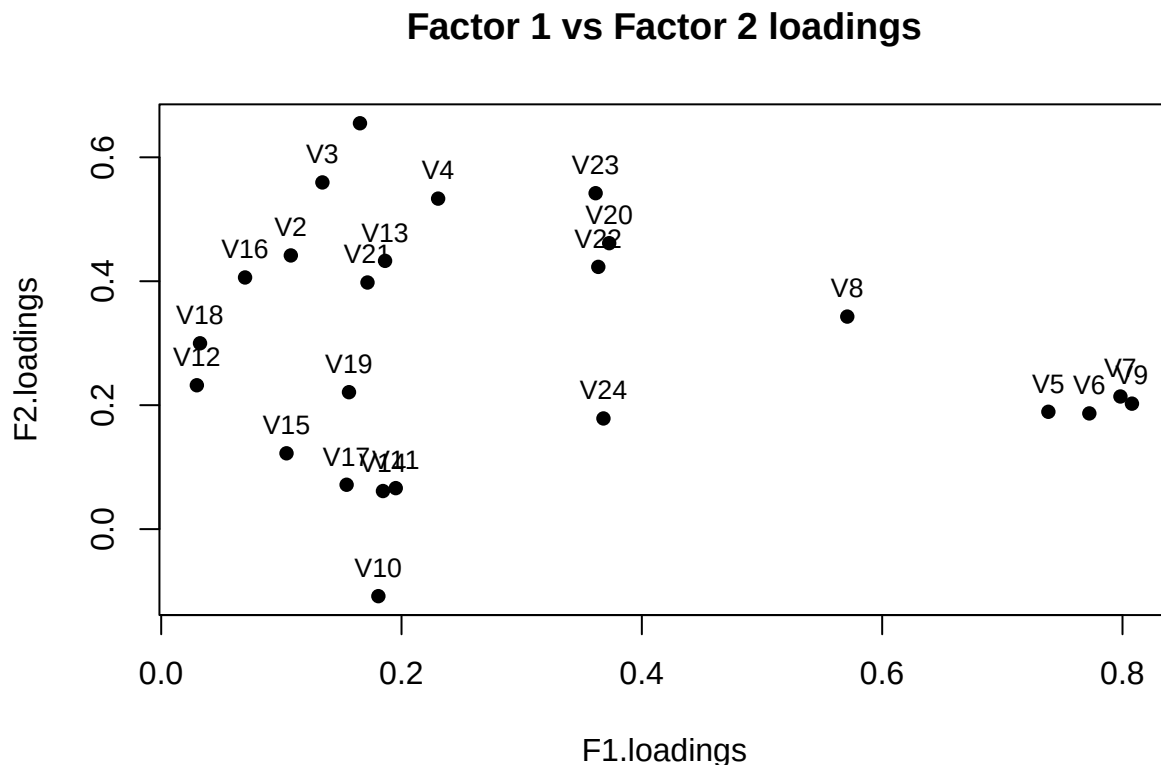
```
#or:
fa.ml$r.scores[2]
```

```
## [1] 0.07425159
```

We notice that their correlation is almost zero, which is something we should expect, in the first place because the factors estimation using the maximum likelihood method assumes that the factors are not correlated; in the second place, we could argue that it is low because the two factors try to explain different characteristics of each individual; indeed, this is the case, since above we were clearly able to distinguish between the first factor, which measures the text comprehension capabilities of each student from the second factor, which explains the creative capabilities of each individual.

Below here we also add the loadings scatterplot of the first two factors:

```
F1.loadings <- fa.ml$loadings[, 1]
F2.loadings <- fa.ml$loadings[, 2]
plot(F2.loadings ~ F1.loadings, pch = 16, main = 'Factor 1 vs Factor 2 loadings')
text(F1.loadings, F2.loadings, colnames(psych.grant[, -c(1, 2)]), cex = .8, pos = 3)
```



We can see from the above that the first two factors have high loadings respectively on variables V5 to V9 for the first one and V1, V3, V4, V23 for the second one, as we observed in the previous point.

Question 1.4

We now do the same we did in point 3, but with the Pasteur students data:

```
psych.pasteur <- psych[psych$group == 'PASTEUR', -27]
fa.ml <- fa(r = psych.pasteur[, -c(1, 2)], nfactors = 5, rotate = 'varimax', fm = 'ml')
```

fa.ml

```
## Factor Analysis using method = ml
## Call: fa(r = psych.pasteur[, -c(1, 2)], nfactors = 5, rotate = "varimax",
##      fm = "ml")
## Standardized loadings (pattern matrix) based upon correlation matrix
##      ML1  ML3  ML4  ML5  ML2  h2  u2 com
## V1  0.31  0.58  0.14  0.10  0.07  0.47  0.53  1.8
## V2  0.04  0.52 -0.01  0.06 -0.14  0.29  0.71  1.2
## V3  0.10  0.44 -0.18  0.00  0.10  0.25  0.75  1.5
## V4 -0.02  0.67  0.19  0.17  0.01  0.52  0.48  1.3
## V5  0.81  0.04 -0.07  0.10  0.14  0.69  0.31  1.1
## V6  0.78  0.16  0.10  0.06  0.20  0.69  0.31  1.3
## V7  0.90  0.08  0.01  0.11 -0.06  0.84  0.16  1.1
## V8  0.68  0.17  0.14  0.15  0.01  0.54  0.46  1.3
## V9  0.78  0.25  0.04  0.10  0.16  0.70  0.30  1.3
## V10 0.14 -0.21  0.12  0.50  0.64  0.74  0.26  2.3
## V11 0.35  0.07  0.23  0.67  0.06  0.63  0.37  1.8
## V12 0.08  0.10 -0.01  0.53  0.22  0.34  0.66  1.5
## V13 0.07  0.27  0.09  0.54  0.05  0.38  0.62  1.6
## V14 0.04  0.05  0.69 -0.03  0.04  0.48  0.52  1.0
## V15 -0.13  0.13  0.61 -0.11  0.10  0.43  0.57  1.3
## V16 0.08  0.39  0.48  0.18  0.16  0.44  0.56  2.6
## V17 0.07 -0.05  0.52  0.29  0.09  0.37  0.63  1.7
## V18 0.10 -0.01  0.46  0.09 -0.01  0.23  0.77  1.2
## V19 0.07  0.24  0.36  0.24  0.03  0.25  0.75  2.7
## V20 0.12  0.51  0.19  0.01  0.10  0.32  0.68  1.5
## V21 0.28  0.39  0.14  0.20  0.43  0.48  0.52  3.4
## V22 0.47  0.48  0.03  0.15  0.05  0.48  0.52  2.2
## V23 0.36  0.59  0.14  0.09  0.30  0.59  0.41  2.4
## V24 0.22  0.29  0.23  0.24  0.53  0.52  0.48  2.9
##
##
##      ML1  ML3  ML4  ML5  ML2
## SS loadings      3.94 2.81 2.02 1.69 1.20
## Proportion Var    0.16 0.12 0.08 0.07 0.05
## Cumulative Var    0.16 0.28 0.37 0.44 0.49
## Proportion Explained 0.34 0.24 0.17 0.14 0.10
## Cumulative Proportion 0.34 0.58 0.75 0.90 1.00
##
## Mean item complexity = 1.8
## Test of the hypothesis that 5 factors are sufficient.
##
## df null model = 276 with the objective function = 10.36 with Chi Square = 1514.85
## df of the model are 166 and the objective function was 1.19
##
## The root mean square of the residuals (RMSR) is 0.03
## The df corrected root mean square of the residuals is 0.04
##
## The harmonic n.obs is 156 with the empirical chi square 101.18 with prob < 1
## The total n.obs was 156 with Likelihood Chi Square = 170.44 with prob < 0.39
##
## Tucker Lewis Index of factoring reliability = 0.994
## RMSEA index = 0.011 and the 90 % confidence intervals are 0 0.039
## BIC = -667.84
```

```
## Fit based upon off diagonal values = 0.98
## Measures of factor score adequacy
##
## Correlation of (regression) scores with factors    ML1  ML3  ML4  ML5  ML2
## Multiple R square of scores with factors          0.96 0.90 0.86 0.84 0.82
## Minimum correlation of possible factor scores      0.91 0.80 0.74 0.70 0.68
## Minimum correlation of possible factor scores      0.83 0.61 0.47 0.40 0.35
```

Analyzing the loadings of each factor, we end up with the following conclusions:

- We see that for the first factor, the loadings are higher for variables V5 to V9, that are the same of the data for the Grant-White school students; we can confirm thus that this factor explains the *text comprehension level* of students;
- On factor 3, the relevant variables are the same of the Grant-White data, with the addition of V20, that is, deduction. Thus, we can more or less confirm the meaning that it has for the other data, i.e., *creative capabilities*;
- From now on we have some differences in the order of the factors: we notice immediately that now ML2 is the factor that explains the less part of variance of the 5; we have in order that the third factor is ML4, on which load highly again the variables V14 to V17, with the exception that V16 has a loading of 0.48, but we include it anyway for completion. Hence, again factor 4 explains the *distinction and pattern recognition* capabilities of the individual
- On factor 5 (ML5) load highly variables V10 to V13, which are respectively: V10 addition, V11 code, V12 counting dots, V13 straight-curved capitals. Thus it seems again that ML5 tries to explain the *analytical capabilities* of the individuals, with the difference that this time the meaning of this factor is clearer than the one it assumes with the Grant-White data because more variables load in a significant way on it;
- On ML2 load high enough variables V10 and V24, which are respectively addition and arithmetic problems. Thus we confirm that factor 2 again tries to explain the *mathematical and logical skills* of the individual, even if on the Grant-White data ML2 we also had that variable V12 (counting dots) loaded high enough on this factor. As we said earlier, the other difference is that ML2 this time explains far less variance than for the other data frame: probably this is because some variable partially tries to explain more or less the same skills: for example, in this case we have that factor 5 has high loadings for V10 (addition) and V11 (code), which are more than analytical, mathematical/logical skills. This hypothesis is confirmed by the fact that, V10 loads for more than 0.5 both on ML5 and ML2 this time, making ML2 explain less information.

Summing up, we can conclude that the factors meaning remained the same more or less, with some minor variation, which however bring to the same overall interpretation of the factors.

Exercise 2

```
library(MASS)
pendigits<-read.table("data/pendigits.txt", sep=",",head=F)
names(pendigits)<-c(paste0(rep(c("x","y"),8),rep(1:8,each=2)),"digit")
```

Question 2.1

We now plot in a scatterplot the first two discriminant variables and their centroids for each class.

```
#Model estimation
lda.fit <- lda(digit ~ ., data = pendigits)
lda.pred <- predict(lda.fit)

#Extracting LDs
LD1 <- lda.pred$x[, 1]
LD2 <- lda.pred$x[, 2]
```

```

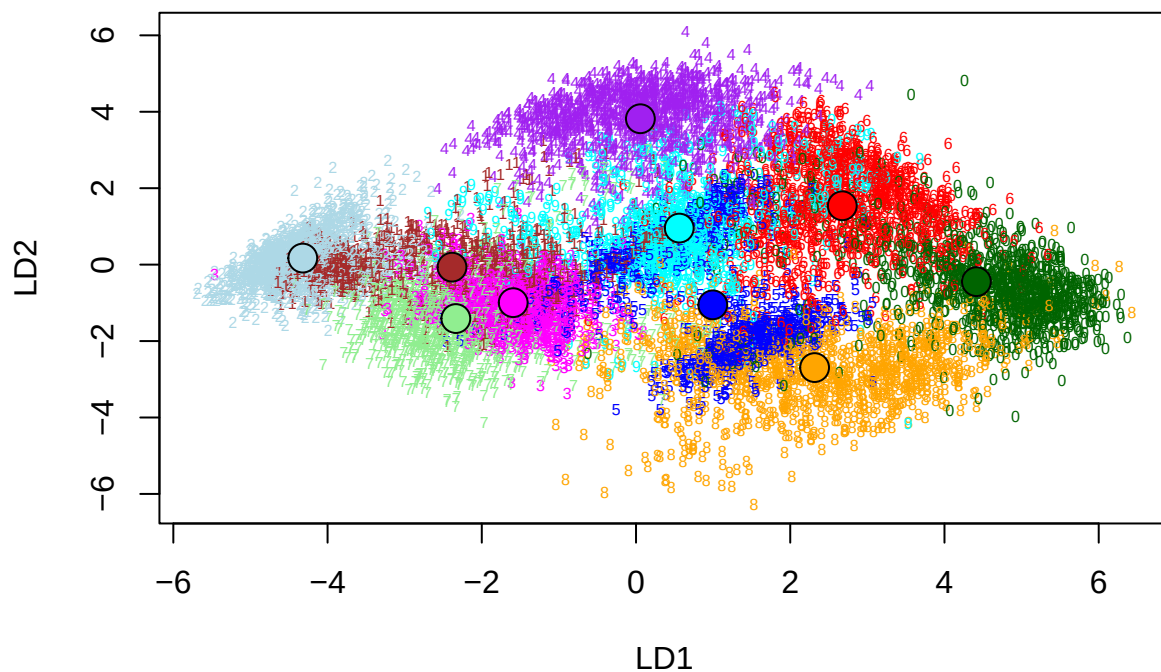
#Centroid computation
LD.centroids <- matrix(0, nrow = 2, ncol = 10)
colnames(LD.centroids) <- 0:9
row.names(LD.centroids) <- c('LD1', 'LD2')
for (i in 0:9) {
  LD.centroids[1, i+1] <- mean(LD1[pendigits$digit == i])
  LD.centroids[2, i+1] <- mean(LD2[pendigits$digit == i])
}

#Creating colors vector
lookup<-c("darkgreen", "brown", "lightblue", "magenta", "purple",
          "blue", "red", "lightgreen", "orange", "cyan")
col.ind <- rep('black', nrow(pendigits))
pch.ind <- rep(0, nrow(pendigits))
for (i in 0:9) {
  col.ind[pendigits$digit == i] <- lookup[i + 1]
  pch.ind[pendigits$digit == i] <- i + 48
}

#Plotting
plot(LD2 ~ LD1, col = col.ind, pch = pch.ind, cex = .5,
     main = 'Scatterplot of the first two linear discriminants')
points(x = LD.centroids[1, ], y = LD.centroids[2, ], pch = 21, cex = 2, bg = lookup)

```

Scatterplot of the first two linear discriminants



We see that most of them are well spread out except for digits 1 and 5, which are particularly hidden by other

classes. The plot we are looking at is an informative low-dimensional projection of the data, made up by the observations transformed in such a way to maximize the distance between each class. A pretty intuitive way of understanding what LDA does is to pose the problem that Fisher posed to himself in order to achieve his goal of developing a classification technique: find a linear combination $Z = a^T X$ such that the between-class variance is maximized relative to the within-class variance. After having solved this maximization problem, we classify each observation x by taking $z = a^T x$ and checking which one of the $\hat{z}_k$ centroids in the transformed space is the closest to z . It turns out that solving this problem translates in reducing the dimensionality of the data to $\min\{K - 1, p\}$ (in this case, we end up having 9 dimensions in which consider the data). This comes from the fact that the K centroids in p -dimensional input space lie in an affine subspace of dimension $\leq K - 1$; then, in locating the closest centroid, we can ignore distances orthogonal to this subspace, since they will contribute equally to each group. This allows us to make distance comparisons on this subspace, and moreover, after solving the problem, the linear discriminant variables $Z_1, \dots, Z_{\min\{K-1, p\}}$ we are left with can be used as a visualization tool: what we are looking at is the scatterplot of the first two linear combinations of the data such that the K classes are spreaded out the most. The function `lda()` orders the discriminant variables in a decreasing order of centroid separation, so the scatterplot of the first two LDs is the one in which the classes are more clearly separated.

Question 2.2

We compute the confusion matrix and then compute the number of misclassified observation for each group.

```
#Matrix
conf.mat <- table(estimated = lda.pred$class, true = pendigits$digit)
conf.mat <- addmargins(conf.mat)
conf.mat
```

##	estimated	0	1	2	3	4	5	6	7	8	9	Sum
##	0	1015	0	0	0	0	0	4	1	76	1	1097
##	1	10	799	23	19	0	3	0	61	18	64	997
##	2	0	206	1113	1	1	0	0	16	0	0	1337
##	3	0	16	1	1020	1	58	0	19	10	10	1135
##	4	33	2	0	0	1116	0	3	7	0	20	1181
##	5	0	51	0	0	0	714	5	5	57	12	844
##	6	5	7	0	0	4	3	1029	0	6	1	1055
##	7	0	29	7	13	1	0	0	1015	2	0	1067
##	8	77	0	0	0	0	9	15	6	875	2	984
##	9	3	33	0	2	21	268	0	12	11	945	1295
##	Sum	1143	1143	1144	1055	1144	1055	1056	1142	1055	1055	10992

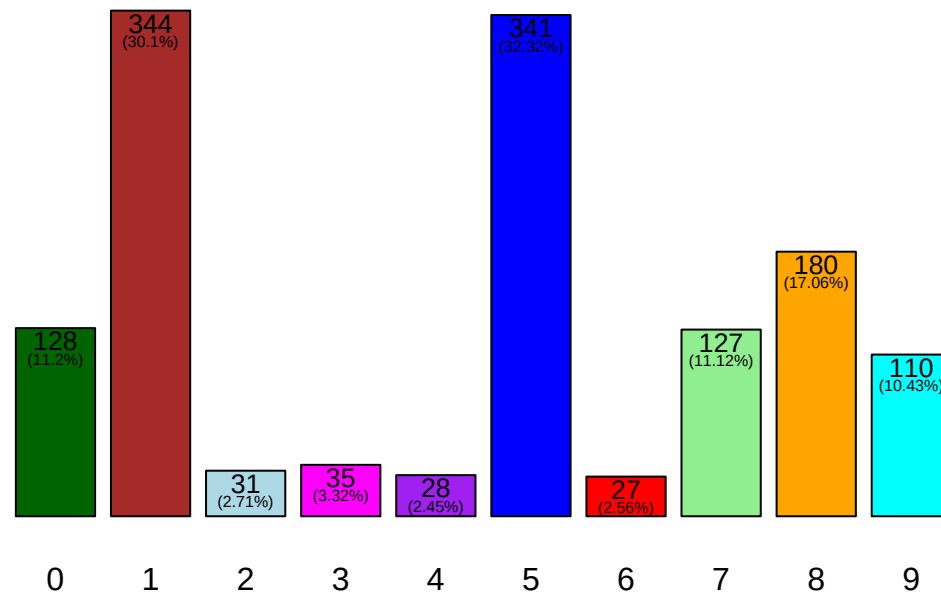
In the diagonal we can see the number of estimated elements correctly classified; outside the diagonal we see the number of element misclassified. To see which is the most difficult to classify correctly, we find the number of misclassifications for each class.

```
#Errors
errors <- rep(0, 10)
names(errors) <- 0:9
for (i in 1:10) {
  errors[i] <- conf.mat[11, i] - conf.mat[i, i]
}

#Plot
bp <- barplot(errors, col = lookup,
               main = 'Number of misclassified observations for each group',
               axes = F, sub = 'Training errors')
text(x = (bp), y = (errors - 9), errors, cex = .8)
```

```
text(x = (bp), y = (errors - 22), paste0('(', round(errors/conf.mat[11, -11]*100,
2), '%', ')'), cex = .5)
```

Number of misclassified observations for each group



Training errors

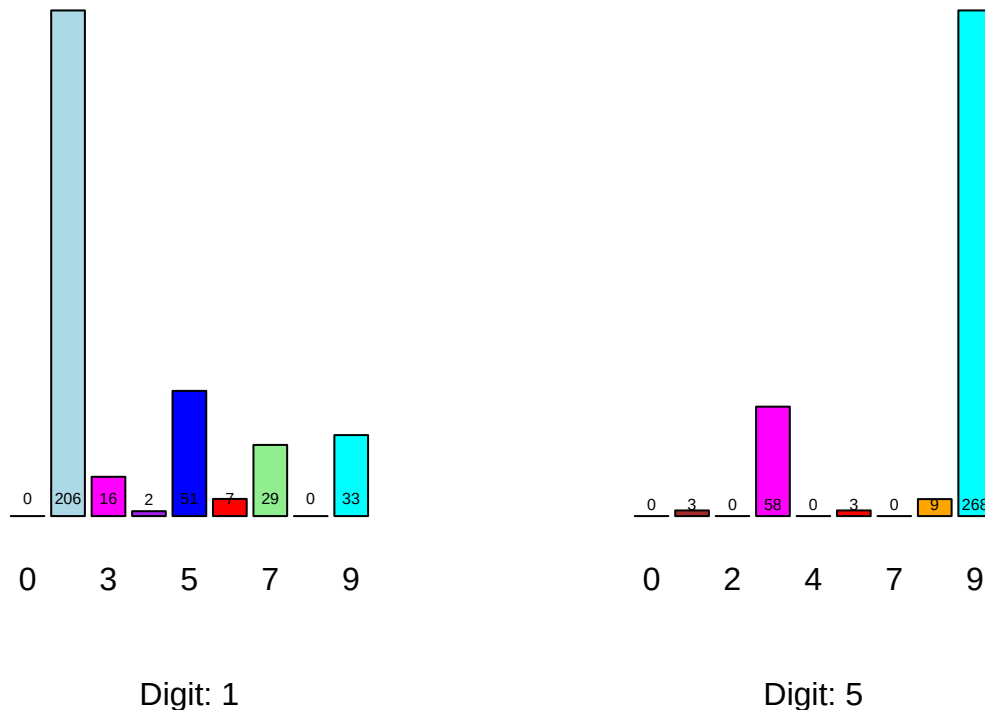
From analyzing the barplot, it is evident that the brown class ($k = 1$) and the blue class ($k = 5$) were the most difficult to identify correctly. This is further supported by the scatterplot of the first two discriminant variables, which shows that these two groups are partially masked by the others. In addition, it's worth mentioning that classes 0, 7, 8, and 9 also have misclassification errors above 10%.

When it comes to digits 1 and 5, we can further investigate which other digits the model is most likely to confuse them with:

```
#Errors vectors
dig.1.mis <- conf.mat[c(1, 3:10), 2]
dig.5.mis <- conf.mat[c(1:5, 7:10), 6]

#Plotting
par(mfrow = c(1, 2))
bp1 <- barplot(dig.1.mis, axes = F, col = lookup[-2], sub = 'Digit: 1')
text(x = bp1, y = 7, dig.1.mis, cex = .5)
bp2 <- barplot(dig.5.mis, axes = F, col = lookup[-6], sub = 'Digit: 5')
text(x = bp2, y = 6, dig.5.mis, cex = .5)
title('Misclassified observations for digits 1 and 5', outer = T, line = -2)
```

Misclassified observations for digits 1 and 5



When reading the barplot, it's important to keep in mind that we have removed the digit of interest: in the first plot, digit 1 is not present, and in the second plot, digit 9 is not plotted.

We notice that most of the errors for the true digit 1 are made by the model that incorrectly classifies it as digit 2 ($\approx 18\%$ of the times). Similarly, the model wrongly classifies digit 5 mostly as digit 9 ($\approx 25\%$ of the times). These misclassifications can be understandable as sometimes people tend to write the digit '1' similarly to '2' and the digit '5' similarly to '9'.

```
#Training error
mean(lda.pred$class != pendigits$digit)
```

```
## [1] 0.1229076
```

It's important to keep in mind that the training error rate tends to underestimate the Actual Error Rate (AER), which is a measure of how well a sample classification rule will perform in future samples. To address this issue, there are two common approaches that can be used to estimate the AER. One method involves using a subset of the total data (training data) to train the model, and then using the remaining data (test data) to evaluate the test error rate, which is an estimation of the AER.

Another approach is the leave-one-out cross-validation (LOOCV) procedure, which is a method for estimating the AER that involves systematically leaving out one observation at a time, using the remaining data to train the model and then test it on the observation removed before. This procedure can provide a more accurate estimate of the AER than the training error rate alone. We will get into the details in the next part.

Question 2.3

We estimate the model using Cross Validation (leave-one-out) and compute errors. We briefly explain how Cross Validation works: we remove from the original data frame k observations, we estimate the model on the remaining part of the data frame (the one we did not remove) and, finally, we test the model on the k observations we removed before. We do this until we have tested the model on all the observations: this

means that we divide the data frame in roughly equal k parts, we train on $n - k$ observations (n is the number of total observations) and we test on the removed k observations each time. The type of CV we are going to apply now is called Leave-One-Out CV simply because we set $k = 1$. Applying this technique on R translates in adding the argument `CV = T` in the `lda()` function. After we are done, we compute the CV error (our AER estimate) as we did above.

```
#Estimation (CV)
lda.fit <- lda(digit ~ ., data = pendigits, CV = T)

#Confusion matrix
conf.mat.cv <- table(estimated = lda.fit$class, true = pendigits$digit)
conf.mat.cv <- addmargins(conf.mat.cv)
conf.mat.cv
```

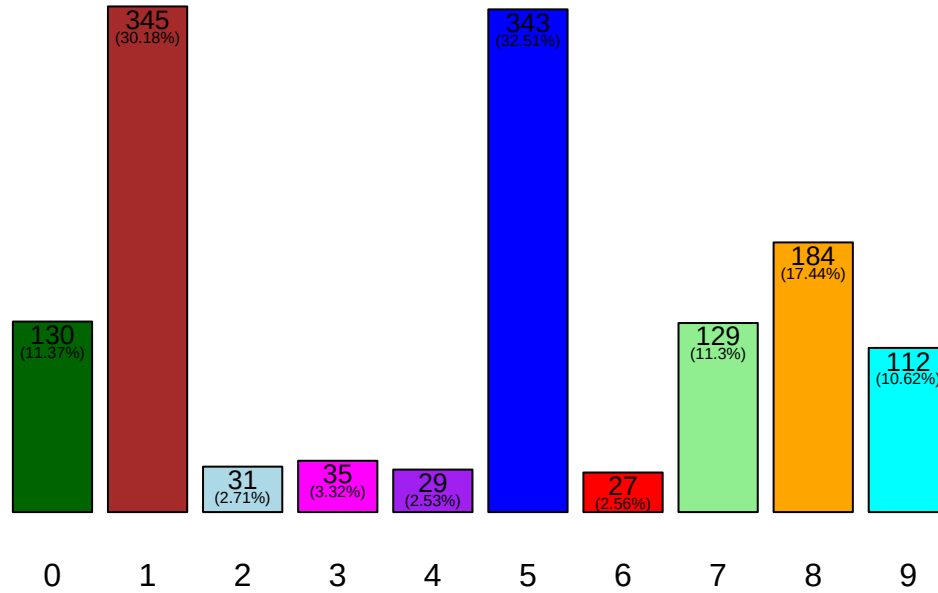
##		true										
##	estimated	0	1	2	3	4	5	6	7	8	9	Sum
##	0	1013	0	0	0	0	0	4	1	79	1	1098
##	1	10	798	23	19	1	3	0	61	18	65	998
##	2	0	206	1113	1	2	0	0	17	0	0	1339
##	3	0	16	1	1020	1	58	0	20	10	10	1136
##	4	34	2	0	0	1115	0	3	7	0	20	1181
##	5	0	51	0	0	0	712	5	5	57	13	843
##	6	5	8	0	0	4	3	1029	0	7	1	1057
##	7	0	29	7	13	1	0	0	1013	2	0	1065
##	8	78	0	0	0	0	9	15	6	871	2	981
##	9	3	33	0	2	20	270	0	12	11	943	1294
##	Sum	1143	1143	1144	1055	1144	1055	1056	1142	1055	1055	10992

We now plot as above the number of misclassified observations for each digit:

```
#Errors
errors.cv <- rep(0, 10)
names(errors.cv) <- 0:9
for (i in 1:10) {
  errors.cv[i] <- conf.mat.cv[11, i] - conf.mat.cv[i, i]
}

#Plot
bp <- barplot(errors.cv, col = lookup,
  main = 'Number of misclassified observations for each group',
  axes = F, sub = 'Cross Validation errors')
text(x = (bp), y = (errors.cv - 9), errors.cv, cex = .8)
text(x = (bp), y = (errors.cv - 22), paste0('(', round(errors.cv/conf.mat[11, -11]*100,
  2), '%', ')'), cex = .5)
```

Number of misclassified observations for each group



Cross Validation errors

```
#Cross Validation error
mean(lda.fit$class != pendigits$digit)
```

```
## [1] 0.1241812
```

We notice that the CV error is slightly higher than the training error (0.1229 vs 0.1242 respectively): this is because the training error usually tends to underestimate the Actual Error Rate (AER); thus, in reality, what we got with Cross Validation is a better estimate of this error because it is nearer to the real one and, since they are pretty similar, it is not that bad as classification error on new observations. Moreover, we see that again the classes most difficult to classify were digits 1 and 5; from the confusion matrix, we also notice that these two digits were misclassified with the same wrong digits almost in the same way as in the case for the training confusion matrix.

Question 2.4

We perform 44-fold Cross Validation on the data; notice that this is equivalent to train the model on 249 writers and to test it on the remaining one, each time. We then combine all the 44 misclassification errors with the following formula:

$$H\text{-fold Cross Validation Error} = \sum_{h=1}^H \frac{n_h}{n} Err_h$$

where H is the number of folds, n_h are the observations in fold h , n the total observations and $Err_h = \frac{1}{n_h} \sum_{i \in C_h} \mathbf{1}(y_i \neq \hat{y}_i)$, that is, the mean of misclassified observations for each test partition (each of the h folds) C_h .

```

#Partitioning
groupCV <- rep(1:44, each=250)
groupCV <- groupCV[1:length(pendigits$digit)]

#Since the last partition we have 242 obs. and not 250, we account for that doing so:
length(groupCV[groupCV == 44])

## [1] 242

part_dim <- rep(250, 43)
part_dim[44] <- 242

#We have K = 10 classes, so we can perform the reduced rank LDA on K-1 = 9 dimensions
K_1 <- 9
misc <- 0
CV_err <- 0
for (j in 1:K_1) {
  for (i in 1:44) {
    train <- pendigits[groupCV != i, ]
    test <- pendigits[groupCV == i, ]

    lda.fit <- lda(digit ~ ., data = train)
    lda.pred <- predict(lda.fit, newdata = test, dimen = j)

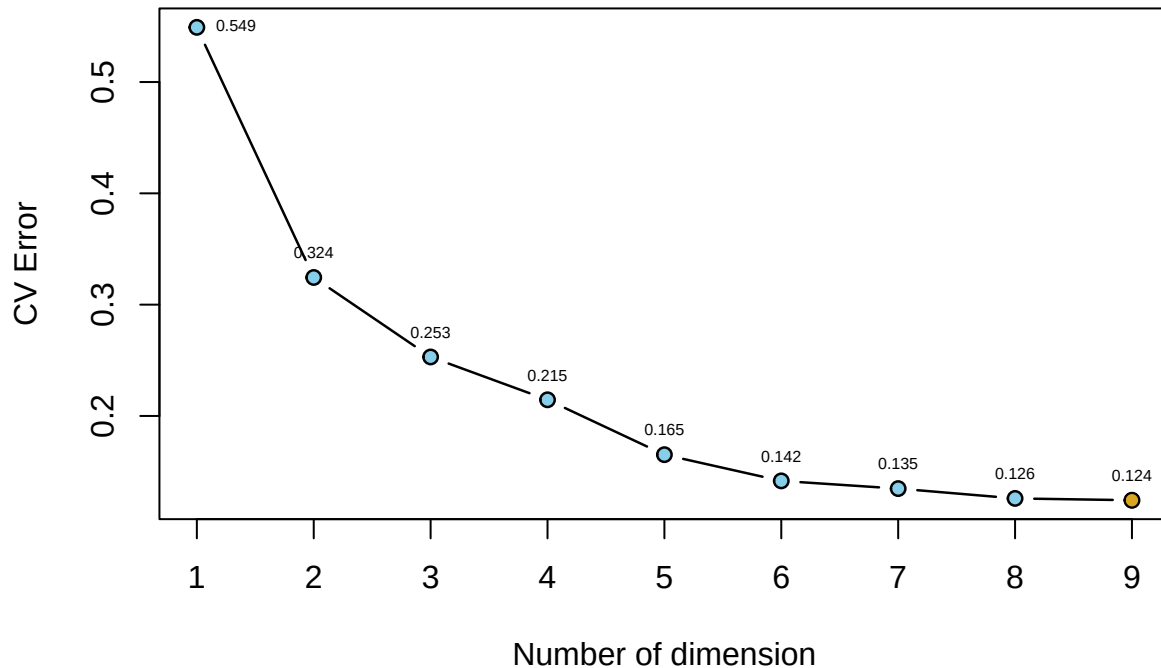
    misc[i] <- mean(lda.pred$class != test$digit)
  }
  CV_err[j] <- sum(part_dim * misc)/nrow(pendigits)
}
plot(CV_err, type = 'b', xlab = 'Number of dimension', ylab = 'CV Error',
     main = 'Cross Validation error for each reduced-rank LDA', pch = 21, bg = 'skyblue',
     axes = F, lwd = 1.25)
text(1:9, CV_err, round(CV_err, 3), cex = .5, pos = c(4, rep(3, 8)))
box()
axis(2)
axis(1, at = 1:9)
which.min(CV_err)

## [1] 9

points(which.min(CV_err), CV_err[which.min(CV_err)], pch = 21, bg = 'goldenrod')

```

Cross Validation error for each reduced-rank LDA



From the plot, the choice is clear: even if after dimension $L = 4$ the error decrease is small, we anyway choose the smallest one, which is obtained by the $L = 9$ LDA, i.e. by the full-rank LDA.

Question 2.5

We will use a technique called Random Forest. We briefly explain how it works and then we apply it.

- First of all, random forests are built upon the simple concept of decision trees, which split data into branches to make predictions. In a decision tree, data is split according to certain conditions or criteria, aiming to organize the data into groups that are as homogeneous as possible at the leaves (the end on the tree). Random forests improve on the performance of a single decision tree by creating a whole forest of them.
- Each tree in the forest is built from a random sample of the data: this sampling is done with replacement, and is known as bootstrap sampling. This is done in order to augment randomness; we have to say that when building each tree, the algorithm uses also other methods to increase randomness even more. The number of trees is pre-specified by us, the more the better in terms of accuracy of final estimates.
- Finally, to make a prediction, a new data point is fed through all the trees in the forest. Each tree gives a vote on what the output should be. For classification, the final prediction is typically the majority vote across all trees (i.e., the most common class predicted by the trees). For regression, it is usually the average of the outputs predicted by all the trees.

Now we can move on the the application to our data frame; we will first estimate the model on some train observations and then use the test observations to get the test error; notice we are not using Cross Validation. To split the data, we create a vector with roughly 75% of elements as 0s (used for train) and roughly 25% as 1s (test observations).

```
library(randomForest)
```

```

#Train and test obs.
sampling.vector <- sample(x = c(0, 1), size = nrow(pendigits), replace = T,
                          prob = c(.75, .25))
train <- pendigits[sampling.vector == 0, ]
test <- pendigits[sampling.vector == 1, ]

#Estimating
fit.rf <- randomForest(factor(digit) ~ ., data = train)

#Testing
test.results <- predict(fit.rf, test)

#Test Error & Confusion Matrix
mean(test.results != test$digit)

```

```
## [1] 0.007489301
```

```
table(estimated = test.results, true = test$digit)
```

```
##          true
## estimated  0  1  2  3  4  5  6  7  8  9
##          0 278  0  0  0  0  0  0  0  0  0
##          1  0 298  0  0  0  0  0  0  0  1
##          2  0  3 281  0  0  0  0  1  0  0
##          3  0  1  0 253  0  2  0  0  0  0
##          4  1  0  0  0 293  0  0  0  0  0
##          5  0  0  0  0  0 260  1  0  0  0
##          6  0  0  0  0  0  0 282  0  0  0
##          7  0  1  0  1  0  0  0 287  2  1
##          8  1  0  0  0  0  0  0  0 267  0
##          9  0  0  0  0  1  2  2  0  0 284
```

As we can see, the error is way lower than it was before for training error, LOOCV error and 44-fold CV. We can see this also by the confusion matrix, which is almost perfectly diagonal.